



ENGINEERING WHITE PAPER

Factoids, Not Chunks

Inside UnicornIQ, the verified knowledge engine for enterprise AI

June 2026 Version 1.0 UnicornIQ.ai

Contents

- 01** Overview: how to read this paper
- 02** The problem, in plain terms
- 03** What UnicornIQ does
- 04** What changes for the business
- 05** Architecture
- 06** Ingestion: from documents to judged factoids
- 07** Trust mechanics: truthfulness, confidence, contradiction
- 08** Retrieval and the API
- 09** Operations, reliability, and security
- 10** What UnicornIQ is for, and what it is not
- 11** Working with UnicornIQ

➤ Overview: how to read this paper 01

HOW TO READ THIS PAPER

This paper is deliberately built in two halves. Sections 2 through 4 are written in plain business language: the problem, what UnicornIQ does about it, and what changes for the organization that adopts it. Sections 5 through 10 are written for engineers and architects, at the level of schemas, pipelines, and operational behavior. Read the first half to decide whether this matters to you. Read the second half to verify that it works the way the first half claims.

One sentence before either half: UnicornIQ is an API-first verified knowledge engine. It ingests your documents, breaks them into small verified facts, scores its confidence in each one, detects contradictions between them, and serves the result to your AI systems over a REST API. It is not a chatbot, it is not a retrieval-augmented generation system, and it is not a wrapper around someone else's model. It is the layer that makes the rest of your AI stack tell the truth.

➤ The problem, in plain terms 02

AI assistants answer from whatever text they are handed at the moment of the question. That single fact explains most of what goes wrong with enterprise AI.

The standard approach retrieves passages of documents, usually called chunks, and trusts the model to sort out the truth on the fly, every time, under a deadline of seconds. But enterprise knowledge does not cooperate. Documents disagree with each other. Policies go stale and nobody removes the old version. The same fact appears five times in five wordings, three of them outdated. The model receives all of it at once and does its best.

The result is the failure mode everyone has now seen firsthand: a confident answer that is wrong, with no way to show where it came from or to prove it either way. For a demo that is embarrassing. For a regulated business it is disqualifying.

The problem is not the model. The problem is what the model is given to read.

➤ What UnicornIQ does 03

UnicornIQ moves the work of verification from query time to ingestion time, and changes the unit of knowledge from a passage of text to a single verified fact.

When a document enters UnicornIQ, it is decomposed into factoids: atomic statements, each one small enough to be individually true or false. Each factoid is judged, given a confidence score, checked against the knowledge already in the library, and stored with a complete record of where it came from. All of that happens once, at ingestion, before any assistant or agent ever uses the knowledge.

At query time there is no improvisation. An application asks a question over the REST API and receives the verified factoids that match, along with their verification state. The application, not UnicornIQ, decides what to do with them. UnicornIQ is a data provider: it does not chat, it does not generate prose, and it does not wrap your query in a prompt of its own.

- **Factoids, not chunks.** The unit of knowledge is a single verifiable statement, not a thousand characters of surrounding text.
- **Verified at ingestion.** Judgment, confidence scoring, and contradiction checking happen before the knowledge is ever served, not in the middle of answering a question.
- **Served over an API.** Any assistant, agent, or workflow can draw on the same verified library through the same REST contract.

➤ What changes for the business 04

Three things change when AI draws on verified facts instead of raw text: answers become provable, knowledge maintains itself, and the economics of every query improve.

Provable answers

Every factoid carries its provenance: the source it came from, every change to its status, who or what made each change, when, and the stated reason. When an answer is challenged, the chain from answer to source is a lookup, not an investigation. For compliance and audit work this is the difference between an AI you can deploy and one you cannot.

Knowledge that maintains itself

When a newer source contradicts an older one, the older factoid is superseded: marked invalid, kept in the library for the record, and removed from answers. Nothing is silently deleted and nothing stale lingers in responses. The library converges on current truth and keeps the receipts.

Better economics on every query

Models are priced by what they read and what they write. A system that hands the model a short list of verified facts instead of a stack of overlapping passages sends fewer tokens in, and because the input arrives already resolved and de-duplicated, the model has less to reconcile and returns shorter, more direct completions. Less in, less out, and a far higher share of what the model reads actually bears on the question.

The remainder of this paper explains how this is built.

➤ Architecture 05

UnicornIQ is a service-oriented system: the useful discipline of services without the sprawl of fifty of them. The services are Python, where AI and ML engineering lives.

One store, two roles

PostgreSQL serves as both the vector store and the relational store. Embeddings, verification status, and provenance live in one place, queried together, with no synchronization seam between a vector database and a system of record. The choice is also economic: dedicated vector databases tend to price by the row, and an architecture that turns one source passage into many small factoid rows would pay that price many times over. Postgres does not care.

The factoid record

The content of a factoid is immutable. It is written once, embedded once, and never moves in vector space. Status and metadata change; the statement itself does not. This single constraint removes an entire class of consistency problems, and it makes the factoid effectively content-addressable: the statement is its own stable identity in the library.

- **Content:** the atomic statement, immutable from the moment of ingestion.
- **Embedding:** computed once at ingestion and fixed thereafter.
- **Source reference:** the document and location the statement was extracted from.
- **Status:** the current verification state, such as valid, invalid, or needs review.
- **Truthfulness and confidence:** the judgment and the system's confidence in that judgment, held as separate measures.
- **Change log:** an append-only history of every status transition, with actor, timestamp, and stated reason.

➤ Ingestion: from documents to judged factoids 06

Ingestion is an AI-assisted ETL pipeline: documents in, judged factoids out. It is where almost all of the system's heavy computation happens, by design.

Source material is decomposed into atomic statements, each statement is embedded, and each is then judged. Judgment assigns the verification status and the confidence score, and checks the new statement against what the library already holds. Every decision the AI makes in this pipeline is required to record a rationale alongside the decision itself. A stated reason is not proof, but it is a permanent, reviewable record that a human can audit, and it enters the same change log as everything else.

Human review runs through the same machinery. When a reviewer confirms, corrects, or invalidates a factoid, their action and their stated reason append to the log exactly as the AI's do. One history, regardless of who wrote it.

The extraction and judgment prompts are standardized engineering assets, versioned and tuned deliberately rather than improvised per request. Consistency at ingestion is what makes the library behave predictably at query time.

➤ Trust mechanics: truthfulness, confidence, contradiction 07

UnicornIQ deliberately separates two questions that most systems blur together: how true is this statement, and how sure are we of that judgment?

Two scales, not one

Consider a statement ingested from a trusted source that nothing in the library contradicts, but nothing corroborates either. It is judged true, and the confidence in that judgment is honest about how much support it has. A statement confirmed by several independent sources earns the same truthfulness with far higher confidence. Collapsing those two situations into one number throws away exactly the information a downstream application needs in order to decide how to act.

The model's own assessment is one input to confidence, not the whole of it. The design direction is a pair of sliding scales in which human review and corroboration across sources adjust the score through a defined calculation, so confidence reflects the evidence rather than a single opinion.

Contradiction by supersession

When a newly ingested statement conflicts with an existing factoid, the system resolves the conflict by supersession. Suppose an older document permits a range of options and a newer one restricts the choice to a single option. The older factoid is marked invalid, the new one enters as valid, and the log records what superseded what and why. The invalid factoid remains in the library: out of every answer, but fully available to anyone auditing how the truth changed and when.

Nothing is deleted. Things are retired, on the record.

➤ Retrieval and the API 08

UnicornIQ answers queries as a data provider over REST. A caller sends a query and receives matched factoids with their verification state. What happens next belongs to the caller.

Retrieval is similarity search over the factoid embeddings, filtered by score. Because each factoid is small and self-contained, matching is precise: the system returns the statements that answer the question, not the passages those statements happen to live in. Determinism comes from the tight scope of a verified library rather than from prompt engineering at query time, because there is no prompt at query time.

When the library holds nothing that supports an answer, the correct behavior is to return nothing. An empty result from a verified library is information; an improvised answer is a liability. Applications built on UnicornIQ inherit that honesty by default.

The API contract is intentionally narrow so it can grow without breaking. Keyword search alongside vector search, re-ranking of results, and richer filtering by status or confidence all fit behind the same interface as the ecosystem around the library develops.

➤ Operations, reliability, and security 09

Each service in the system scales horizontally on its own. The heavy load is ingestion; the query path is deliberately light.

Built for unreliable dependencies

The first thing to strain under load is not UnicornIQ. It is the throughput of the external model providers everyone now shares. The system is built on the assumption that anything it did not build may be unavailable at any given moment: requests retry with backoff, models can be switched, and every interaction with an external model is confined to the background ingestion pipeline. A slow provider makes ingestion take longer. It does not touch the query path, where the work is similarity search and relational lookups against the local store.

Model strategy

UnicornIQ is configured to work across multiple frontier model providers, with Google Gemini and Anthropic Claude models supported today. There is no fine-tuning and no post-training modification; the system's behavior comes from disciplined prompt engineering over well-structured tasks. Extraction and judgment over well-formatted input do not require the largest models available, which keeps unit cost down and keeps the door open to hosting open-weight models where throughput or service-level control demands it.

Tenancy and secrets

Each customer's deployment is isolated at the cloud project boundary on Google Cloud Platform. Authentication and role-based access control run through Auth0. Credentials and keys are held in GCP Secret Manager with per-tenant key isolation, so one tenant's secrets are cryptographically separated from every other tenant's.

➤ What UnicornIQ is for, and what it is not 10

UnicornIQ is built for stable, governed knowledge: the facts an organization needs to get right, prove, and keep current.

Policy, compliance, regulation, product truth, institutional knowledge: this is the core territory. It includes time-aware questions that ordinary retrieval cannot answer cleanly, such as what the rule was at the date of the last audit and whether the requirement was met at that time. The provenance log and the supersession model exist precisely to answer that class of question.

It is not built for live operational state. An inventory count, an order status, a sensor reading: those belong to the system of record that owns them, answered by a direct query, not routed through a knowledge layer that adds a hop and a model. UnicornIQ sits beside the systems of record, not in front of them, and the boundary is deliberate.

➤ Working with UnicornIQ ¹¹

UnicornIQ integrates wherever a REST API reaches: existing assistants, agent frameworks, internal tools, and customer-facing applications. The practical starting point is a single corpus that matters, ingested and verified, with one application drawing on it. From there the library grows and every application that connects to it inherits the same verified truth.

Organizations that want a complete, hosted application on top of the engine should read the companion paper on Vera, the AI front door built on UnicornIQ.

UnicornIQ.ai

© 2026 UnicornIQ, LLC. UnicornIQ, Vera, and the UnicornIQ logo are trademarks of UnicornIQ, LLC. All other marks are the property of their respective owners. UnicornIQ, LLC, 6917 S Louise Avenue, Sioux Falls, SD 57108.